



Introducción a los Métodos Ágiles

Contenidos

Objetivos de la Unidad 4	2
Recursos adicionales	2
Introducción a las Metodologías Ágiles	3
Procesos Predecibles vs. Adaptables	3
La Importancia del Factor Humano en los Métodos Ágiles	5
Los Métodos Ágiles	5
Introducción al Modelado Ágil (AM)	8
Objetivos del AM	8
Alcances del AM	8
Visión general de los Valores, Principios, y Prácticas de AM	8
Valores del AM	10
Principios del AM	10
Prácticas del AM	14
Integración entre las prácticas de AM	17
Desarrollo Ágil de Software	20
Ciclo de Vida para el Desarrollo Ágil de Software	20
Integración de AM con XP (Programación Extrema)	21
Integración de AM con RUP (Proceso Unificado de Rational)	22
Una Introducción a XP	26
¿Qué es la programación Extrema?	26
Prácticas Centrales de XP	27
El Proyecto XP	29
Límites de XP	31
Conclusiones	33

Objetivos de la Unidad 4

En esta unidad presentaremos una introducción a uno de los enfoques metodológicos más modernos y controvertidos: los Métodos Ágiles. Aparecieron a fines del siglo XX como una reacción frente a las metodologías tradicionales por su burocracia. Rápidamente ganaron adeptos, y generaron opiniones diversas, desde seguidores fervorosos a detractores y enemigos.

El objetivo nuestro será presentar las ideas y principios generales de estos métodos evitando dar una opinión determinante sobre su conveniencia o no. En el futuro el alumno mismo podrá ensayar su práctica y evaluar por sí mismo su conveniencia o no.

Los objetivos específicos de esta unidad son:

- Que el alumno conozca y comprenda los problemas de los ciclos de vida y metodologías tradicionales.
- Que el alumno conozca y comprenda los objetivos generales, valores, principios y prácticas fundamentales de los métodos ágiles de desarrollo.
- Que el alumno conozca y comprenda la metodología de desarrollo de Programación Extrema (XP).

Recursos adicionales

Bibliografía sugerida:

- EXTREME PROGRAMMING EXPLAINED: EMBRACE CHANGE
Kent Beck
Adison-Wesley
- AGILE SOFTWARE DEVELOPMENT
Alistair Cockburn

Sitios en la Web

<http://www.agilemodeling.com/>
<http://martinfowler.com/articles/newMethodology.html>
<http://martinfowler.com/articles/agileStory.html>
<http://martinfowler.com/articles/designDead.html>
<http://www.agilealliance.org/home/>
<http://agilemanifesto.org/>
<http://www.extremeprogramming.org/>
<http://www.xprogramming.com/>
<http://www.computer.org/SEweb/Dynabook/>
<http://www.sdmagazine.com/documents/s=844/sdm0108a/0108a.htm>

Introducción a las Metodologías Ágiles

Durante muchos años la actividad de desarrollo de software fue una actividad caótica caracterizada por el desarrollo artesanal y la falta de procesos formales y establecidos. Esta situación es caracterizada por la frase “codificar y corregir”.

Como reacción a esta situación, se desarrollaron diversas metodologías clásicas para formalizar el proceso de desarrollo de software. Estas metodologías impusieron un proceso disciplinado en la actividad de desarrollo de software con el objetivo de hacer este proceso más predecible, reproducible, y con mayor eficiencia. Esto se hizo estableciendo un proceso detallado con un fuerte énfasis en la planificación inspirado en otras disciplinas de la ingeniería.

Sin embargo, estas metodologías no han probado ser notablemente exitosas ni alcanzar los objetivos propuestos, ni tampoco han logrado ser notoriamente populares. La crítica mas habitual a las metodologías clásicas es sobre su burocracia.

Como reacción a estas metodologías clásicas, recientemente en lo últimos años han aparecido un grupo de metodologías conocidas como métodos livianos, o por el término más actual de “metodologías ágiles”. Estos métodos buscan una solución de compromiso entre la ausencia total de procesos formales, y el exceso de procesos demasiado burocráticos y “pesados”.

Una de las diferencias inmediatas de los métodos ágiles es que son menos orientados a la documentación, usualmente enfatizan en el uso de una menor cantidad de documentación, y están más orientados al código al cual consideran una parte clave de documentación.

Las diferencias fundamentales entre los métodos ágiles y las metodologías tradicionales pueden establecerse en:

- **Los métodos ágiles son adaptativos en vez de predictivos:** las metodologías clásicas tienden a planear gran parte del proceso de software en gran detalle para un largo período de tiempo. Esto funciona correctamente hasta que se producen cambios. Su naturaleza es resistir el cambio. Los métodos ágiles en su lugar, **intentan abrazar el cambio y adaptarse al mismo.**
- **Los métodos ágiles están orientados a las personas en vez de orientados a los procesos:** Los métodos ágiles ponen un especial énfasis en trabajar con la naturaleza de la gente en vez de trabajar en contra de ella.

Procesos Predecibles vs. Adaptables

Separación del Diseño de la Construcción

Las metodologías clásicas usualmente se inspiraron en disciplinas de ingeniería tradicionales como la ingeniería civil o mecánica. Estas ingenierías ponen un gran énfasis en planificar antes de construir. Los ingenieros producen una serie de modelos suficientemente detallados, como por ejemplo los planos para construir un puente, que indican qué debe construirse y como deben ensamblarse las piezas. Luego estos modelos son utilizados por otro grupo de personas que se encargan de la construcción física. Esto a su vez permite una razonable predicción de la planificación y del presupuesto del proyecto. Los modelos construidos por el diseño a su vez pueden ser validados por el uso de disciplinas formales como el análisis matemático.

Este enfoque claramente produce una marcada separación entre el *Diseño* y la *Construcción*. Normalmente se necesita personal altamente capacitado intelectualmente para realizar el diseño, en tanto que la construcción es realizada por personal con menor capacitación intelectual pero alta destreza manual.

Trasladando este enfoque al desarrollo de software, el plan del diseño podría tomar la forma de un conjunto de diagramas en UML, y debería contemplar todas las decisiones necesarias para que el codificador realice la *construcción*. Sin embargo, lo difícil aquí es que un modelo de diseño sea lo

suficientemente completo y válido como para que la construcción solo radique en la codificación del mismo.

Otra cuestión son los costos comparativos. Por ejemplo, en la construcción de un puente el esfuerzo de diseño es un 10% del trabajo completo, en tanto que en el desarrollo de software el esfuerzo de diseño puede ocupar un 50%.

Estas cuestiones llevan a considerar que de hecho el mismo código fuente es un documento de diseño y que la fase de construcción se limitaría al uso del compilador y el link editor. Por esto, en software el mayor esfuerzo está puesto en el diseño, el cual requiere personal altamente capacitado y creativo. Este proceso no es fácilmente planificable y por tanto, no predecible.

La impredecibilidad de los requerimientos

En la construcción de software de negocios el cambio de requerimientos es la norma, la cuestión es que actitud se toma sobre el mismo.

Una actitud es considerar el cambio de requerimientos como el resultado de una ingeniería de requerimientos pobre. La idea detrás de la ingeniería de requerimientos es tener una comprensión completa de los requerimientos antes de comenzar a construir el software. La idea es resistir al cambio.

La estimación es difícil por muchas razones. En parte porque el desarrollo de software es una actividad de diseño, y por lo tanto difícil de planificar y ponderar. Por otra parte, la misma naturaleza intangible del software dificulta la valorización de una determinada característica.

Surge entonces la pregunta ¿es la predictibilidad de requerimientos algo imposible?. En general no. Hay determinado tipo de software cuyo desarrollo es totalmente predecible, por ejemplo un programa de control para un trasbordador espacial. Sin embargo, esto rara vez se verifica para el software de negocios.

Uno de los mayores peligros es pretender seguir un proceso predecible cuando no se puede aplicar. Los metodólogos, usualmente no identifican correctamente los límites de aplicabilidad de sus métodos. Esto conduce a que las personas intenten aplicar métodos predecibles en situaciones impredecibles.

Esto no significa que deba caerse irreversiblemente en un caos incontrolable. Lo que se necesita es un proceso que pueda brindar control sobre lo impredecible. Sobre esto tratan precisamente los *procesos adaptables*.

¿Cómo podemos entonces controlar un proceso en condiciones impredecibles? Lo más importante y difícil, es tener conocimiento de la situación en que nos encontramos en todo momento. Necesitamos un mecanismo de retroalimentación que nos indique a intervalos convenientes cual es la situación.

La clave para obtener esta retroalimentación es el desarrollo iterativo. Esta no es una idea nueva, y el desarrollo iterativo ha estado en los métodos de desarrollo desde hace varios años con diferentes nombres. La clave para el desarrollo iterativo es producir frecuentemente versiones operacionales del sistema final que implementen un subconjunto de las características deseadas. Cada una de estas versiones, si bien tiene un subconjunto limitado en características finales, ya es una unidad debidamente probada e integrada.

La importancia de esto, radica en que aún una muy buena documentación puede esconder un sin número de errores u omisiones, pero un producto funcionando es lo más apropiado para detectar rápidamente errores del programa y de comprensión de requerimientos.

El desarrollo iterativo es clave en un proceso adaptativo. Esto lleva a un tipo de planificación en donde los planes a largo plazo son muy variables, y solo los planes a corto plazo, que son realizados para una iteración, son estables.

La cuestión clave para esto es cuánto debe durar una iteración? Hay diferentes respuestas: XP sugiere iteraciones de una a tres semanas, SCRUM sugiere una duración de un mes, etc. La tendencia es sin embargo, tener cada iteración lo mas breve posible, ya que esto posibilita una retroalimentación más frecuente.

Este tipo de procesos adaptativo requiere un tipo de relación muy diferente con el cliente. Los contratos de precio fijo requieren requerimientos estables y por lo tanto procesos predictivos. Los

procesos adaptativos y los requerimientos inestables implican la imposibilidad de trabajar con la usual noción de precio fijo.

En un proceso adaptativo el cliente tiene un control más fino sobre el proceso de desarrollo. En cada iteración puede chequear el progreso del desarrollo y alterar el curso del mismo. Esto posibilita una relación mucho más próxima con el desarrollador del software, una verdadera sociedad de negocios. Este nivel de compromiso no es para cualquier organización cliente, no para todo desarrollador, pero si es esencial para que un proceso adaptativo funcione correctamente.

La Importancia del Factor Humano en los Métodos Ágiles

Llevar adelante un proceso adaptativo no es sencillo. En particular se requiere un equipo de desarrolladores altamente efectivo. No solo es importante la calidad de los individuos, sino también la forma en que se integran en el equipo.

Uno de los objetivos de las metodologías tradicionales es desarrollar un proceso donde las personas involucradas sean partes reemplazables. De esta forma se trata a las personas como recursos de distinto tipo (analistas, programadores, etc.). Los individuos no son importantes, sino los roles que desempeñan.

Sobre esto, las metodologías ágiles difieren diametralmente. Para los métodos ágiles, las personas no son recursos reemplazables, y por otra parte constituyen el factor más importante en el desarrollo de software.

Así mismo, los métodos ágiles no adhieren a la visión Taylorista según la cual, las tareas desarrolladas por los trabajadores son planeadas por otras personas intelectualmente más competentes que estudian y dictaminan como deben hacerse las cosas. Esto no es aplicable al trabajo de desarrollo de software que es una tarea altamente creativa. Los desarrolladores de software deben tener la capacidad necesaria para determinar como llevar adelante su trabajo.

Una consecuencia de esto es el cambio en el estilo de gestión de los equipos de trabajo ágiles. En estos no funciona la imposición de procesos rígidos. Por otro lado, importantes decisiones técnicas son desplazadas del administrador del equipo a los miembros del mismo. Por ejemplo la estimación del tiempo necesaria para desarrollar una determinada característica del software, debe ser realizada por las personas mismas que la desarrollarán.

Los Métodos Ágiles

Basados en las ideas expuestas, en los últimos años han aparecido una serie de metodologías que comparten el calificativo de *ágiles*. Entre las más destacadas se encuentran XP (Extreme Programming), Scrum, Crystal, ASD (Adaptive Software Development), FDD (Feature Driven Development), y DSDM (Dynamic System Development Method).

Todas estas metodologías presentan características comunes. En virtud de ello, representantes de cada una de ellas se reunieron en Febrero del 2001 en Snowbird – Utah, para aunar ideas y criterios. Como resultado de este encuentro, firmaron el llamado Manifiesto para el Desarrollo Ágil de Software:

Manifiesto para el Desarrollo Ágil de Software

Hemos descubierto mejores formas de desarrollar software haciéndolo y ayudando a otros a hacerlo. **A través de este trabajo hemos venido a valorizar:**

- **Individuos e interacciones**, por sobre procesos y herramientas,
- **Software funcionando**, por sobre extensa documentación,
- **Colaboración del Cliente**, por sobre negociación contractual,
- **Responder al Cambio**, por sobre seguir un plan.

Esto es, mientras existe valor en los ítems de la derecha, nosotros valorizamos los ítems de la izquierda.

Principios del Manifiesto Ágil

- Nuestra prioridad más alta es satisfacer al cliente a través de la entrega temprana y continua de software valioso.
- El cambio de requerimientos es bienvenido, aún tarde en el desarrollo. Los procesos ágiles aprovechan el cambio para una ventaja competitiva del cliente.
- Entregar software funcionando frecuentemente, desde un par de semanas a un par de meses, con la mayor brevedad preferentemente.
- Las personas del negocio y los desarrolladores trabajan diariamente juntos en el proyecto.
- Construir proyectos alrededor de individuos motivados. Brindarles el entorno y el soporte que necesitan, y confiar en ellos para obtener el trabajo realizado.
- El método más eficiente y efectivo para transmitir información hacia y dentro del equipo de desarrollo, es la conversación cara a cara.
- Software funcionando es la primer medida de progreso.
- Los procesos ágiles promueven el desarrollo sostenible. Los stakeholders, desarrolladores, y usuarios deben ser capaces de mantener un ritmo constante indefinidamente.
- La atención continua a la excelencia técnica y el buen diseño aumenta la agilidad.
- La simplicidad – el arte de maximizar el trabajo no realizado – es esencial.
- Las mejores arquitecturas, requerimientos, y diseños emergen de equipos auto organizados.
- A intervalos regulares, reflexiona como ser más efectivo, y luego ajusta y afina su comportamiento en concordancia.

Basándose en los principios de los métodos ágiles, encontramos la disciplina de *Modelado Ágil* (AM) de Scott Ambler. El objetivo de la misma no es en sí prescribir un método de desarrollo

completo como puede ser XP, sino indicar como aplicar valores, principios, y prácticas ágiles al modelado del software, independientemente de la metodología base que se utilice.

El resto del presente trabajo aborda el desarrollo comprensivo de AM y tópicos sobre su aplicación con métodos ágiles como Programación Extrema (XP), y otros métodos no encuadrados específicamente dentro de los métodos ágiles como el Proceso Unificado de Rational (RUP).

Introducción al Modelado Ágil (AM)

Agile Modeling (AM) es una metodología basada en prácticas para el efectivo modelado y documentación de sistemas de software.

AM es una colección de prácticas guiadas por valores y principios para modelar software, que puede ser aplicados en proyectos de desarrollo de software de una forma efectiva y 'liviana' en el trabajo día a día.

Puede adoptarse un modelado ágil para todas las actividades del ciclo de desarrollo, captura de requisitos, análisis, arquitectura, y diseño.

AM no es un proceso prescriptivo o establecido, ni define procedimientos detallados de cómo crear un determinado tipo de modelo. En su lugar, sugiere recomendaciones sobre como modelar efectivamente. AM no implica modelar menos, de hecho muchos desarrolladores se encuentran que modelan más siguiendo AM que con otros métodos. Debe pensarse en AM más como en un arte que como en una ciencia.

Objetivos del AM

- Definir e indicar como poner en práctica una serie de *valores, principios, y prácticas* que conlleven al modelado efectivo y liviano. El secreto de AM no son las técnicas en si mismo (tales como casos de uso, modelos de clases, modelos de datos, etc.) sino, como dichas técnicas son aplicadas.
- Explorar la aplicación de técnicas de modelado en proyectos de software a través de un enfoque ágil tal como XP, Dynamic Systems Development Method (DSDM), o SCRUM.
- Explorar como mejorar el modelado en procesos prescriptivos, tales como el Proceso Unificado (UP), ya sea en el caso del RUP (Rational Unified Process) o del EUP (Enterprise Unified Process).

Alcances del AM

Un concepto importante a tener en cuenta es que AM no es un proceso de software completo. AM se enfoca en el efectivo modelado y documentación. Esto significa que:

- ◆ No incluye actividades de programación, aunque recomienda probar los modelos con código.
- ◆ No incluye actividades de prueba, aunque recomienda considerar la prueba durante el proceso de modelado.
- ◆ No cubre la administración del proyecto, distribución del sistema, operación del sistema, soporte del sistema, y una gran cantidad de otros aspectos.

Debido a que AM se enfoca en una porción del proceso completo, debe considerarse su uso en el contexto de otro proceso completo como XP, DSDM, SCRUM, o RUP.

Se parte con un proceso base como XP, RUP, o quizá el proceso ya existente en la organización, y se adapta el mismo para adoptar alguna o todas las prácticas de AM, junto con otras técnicas que fueran necesarias para adecuarlo a las necesidades propias del caso.

Visión general de los Valores, Principios, y Prácticas de AM

Los **Valores** de AM, adoptando y extendiendo lo valores de XP, son *comunicación, simplicidad, retroalimentación, coraje, y humildad*.

Las claves para el modelado exitoso son tener una comunicación efectiva entre todos los que soportan el proyecto (stakeholder), esforzarse en alcanzar la solución más sencilla que cubra las necesidades del cliente, obtener rápida retroalimentación de sus esfuerzos de desarrollo, tener el coraje para tomar las decisiones necesarias, y la humildad para reconocer que no se sabe todo y que otros pueden aportar valores al proyecto.

AM se basa en una colección de **principios**, como la importancia de *asumir simplicidad* cuando se modela y *abrazar el cambio* como parte del trabajo ya que efectivamente los requerimientos cambian con el tiempo. Debe reconocer que el *cambio incremental* del sistema a través del tiempo posibilita agilidad, y debe esforzarse en obtener una *rápida retroalimentación* sobre su trabajo para asegurarse de que el mismo cumple con los requerimientos del cliente. Debe *modelar con un propósito*, si no se sabe porque se está trabajando no debería hacerlo. Se necesitan *múltiples modelos* para ser efectivo. Un concepto crítico es que no todos los modelos son documentos necesarios, principio éste que posibilita *viajar liviano* descartando la mayoría de los modelos una vez que han cumplido su propósito. Los modeladores ágiles creen que el *contenido es más importante que la representación*, y existiendo múltiples formas de modelar un mismo concepto debe elegirse la más apropiada. Para ser un modelador ágil debe *conocer sus herramientas* y *conocer sus modelos*, y para ser un buen compañero de un equipo debe considerarse que *todos pueden aprender de todos*, debe *trabajar con los instintos de las personas*, y a menudo la *comunicación abierta y honesta* es la mejor política a seguir para asegurar la efectividad del equipo del trabajo. Finalmente, un foco sobre la calidad del trabajo es importante debido a nadie le gusta producir un mal trabajo, y las *adaptaciones locales* de AM para cubrir sus necesidades son muy importante.

Para modelar de modo ágil deben aplicarse las **prácticas** de AM apropiadamente. Las prácticas fundamentales incluyen la *creación de varios modelos en paralelo*, *aplicación de los artefactos correctos* para cada situación, e *iterar a otros artefactos* para continuar avanzando con paso seguro. *Modelar en pequeños incrementos*, y no intentar crear un modelo que lo contenga todo, es también fundamental para el éxito como modelador ágil. Ya que los modelos son representaciones abstractas del software, abstracciones que pueden no ser exactas, debe esforzarse en *probarlas con código* para ver que sus ideas trabajan en la práctica y no solo en la teoría. La *participación activa de todos aquellos que soportan el proyecto* es crítica para el éxito del esfuerzo de modelado ya que ellos conocen lo que se desea y pueden proveer la retroalimentación que se requiere. Hay dos razones fundamentales por las cuales se crean los modelos, por un lado se *modela para comprender* un determinado aspecto, por otra parte se *modela para comunicar* que es lo que su equipo está haciendo (o ha hecho).

El principio de *asumir simplicidad* es soportado por las prácticas de *crear contenidos simples* enfocándose solo en los aspectos que se necesitan modelar y no intentando crear un modelo altamente detallado, *representando los modelos de manera simple* vía el uso de notaciones simples, y utilizando las *herramientas más simples* para crear sus modelos.

Se *viaja liviano descartando los modelos temporales* y *actualizando los modelos solo cuando sea perjudicial* mantenerlos desactualizado.

La comunicación es facilitada *presentando los modelos públicamente*, ya sea sobre una pared, o en un sitio Web, a través de la *propiedad colectiva* de todos los artefactos del proyecto, a través de la *aplicación de estándares de modelado*, y *modelando con otros*.

Los esfuerzos de desarrollo son notablemente extendidos cuando se *considera la prueba* del sistema, se *aplican patrones cuidadosamente*, y se *reutilizan recursos existentes*.

Ya que a menudo es necesario integración con otros sistemas, se necesita *formalizar modelos contractuales* con los propietarios de dichos sistemas.

AM es una colección de prácticas que reflejan principios y valores compartidos por muchos desarrolladores de software experimentados. Estas prácticas pueden aplicarse a la mayoría de los proyectos de desarrollo de software. No es necesario que se siga un proceso de desarrollo ágil como XP para poder obtener ventaja de los enfoques descriptos por AM.

No es necesario aplicar la totalidad de las prácticas, principios, y valores de AM para obtener beneficios de ella. Es posible adaptar los procesos propios de cada organización para adoptar algunas prácticas de AM, aunque siempre será más exitoso si se adoptan todas las prácticas.

Valores del AM

AM incluye todos los valores de XP, *comunicación*, *simplicidad*, *retroalimentación*, y *coraje*, agregando un quinto valor: *humildad*.

Comunicación:

Los modelos promueven la comunicación entre el equipo y los representantes del usuario, como así también entre los desarrolladores del equipo.

Simplicidad:

Es importante que los desarrolladores comprendan que los modelos son críticos para simplificar tanto el software y como el proceso de desarrollo de software. Es más sencillo explorar una idea y mejorarla a medida que su conocimiento se incrementa, dibujando un diagrama o dos en vez de escribir decenas o centenas de líneas de código.

Retroalimentación:

Comunicando las ideas a través de diagramas puede obtenerse una rápida retroalimentación, posibilitando actuar en consecuencia.

Coraje:

Coraje es importante ya que se necesitan tomar decisiones importantes y estar habilitado para cambiar el rumbo descartando o refactorizando su trabajo cuando algunas decisiones probaron ser inadecuadas.

Humildad:

Los mejores desarrolladores tienen la humildad para reconocer que ellos no saben todo, que los desarrolladores que lo acompañan, los clientes, y todos los participantes del proyecto tienen su propia área de experiencia y tienen valor para el proyecto. Un enfoque efectivo es asumir que todos los involucrados con su proyecto tienen igual valor y por lo tanto deben ser tratados con respeto.

Principios del AM

AM define una colección de principios centrales y adicionales que cuando se aplican en un proyecto de desarrollo de software fijan el escenario para una colección de prácticas de modelado. Algunos de los principios han sido adoptados de XP.

Principios Centrales:

Asumir Simplicidad:

Debe asumirse que la solución más simple es la mejor. No debe sobredimensionarse el software, o en el caso de AM no debe representarse características adicionales en el modelo que no son necesarias hoy. Tenga el coraje de no necesitar sobre-modelar su sistema hoy. Debe basarse el modelo en los requerimientos existentes hoy, y refactorizar el sistema en el futuro cuando los requerimientos evolucionen. Mantenga los modelos tan simples como sea posible.

Abrazar el cambio:

Los requerimientos evolucionan a través del tiempo. La comprensión de las personas de los requerimientos cambia a través del tiempo. Los participantes del proyecto pueden cambiar a medida que se avanza en el proyecto. Los participantes del proyecto pueden cambiar sus puntos de vista, como también cambiar sus objetivos y criterios de éxito para los esfuerzos del proyecto. La consecuencia es que el ambiente del proyecto cambia a medida que avanza su esfuerzo, y como resultado su enfoque de desarrollo debe reflejar dicha realidad.

Posibilitar el siguiente esfuerzo es el objetivo secundario:

Su producto puede ser considerado una falla aun cuando su equipo entregue un sistema trabajando al usuario. Parte del cumplimiento de las necesidades del usuario es asegurar que el sistema es lo suficientemente robusto como para ser extendido a través del tiempo. El siguiente esfuerzo puede ser el desarrollo de una versión mayor de su sistema, o simplemente puede ser la operación y soporte de la aplicación. Para posibilitar esto no solo es importante desarrollar software de calidad sino también crear la documentación suficiente y material de soporte para que las personas involucradas en la próxima etapa del sistema puedan desarrollar su objetivo efectivamente.

Cambio incremental:

Un concepto importante a comprender con respecto al modelado, es que no necesita ser correcto desde el primer momento, de hecho es bastante improbable que así sea. Más aún, no es necesario capturar cada detalle en sus modelos. Solo necesita tener los necesarios al momento. En vez de intentar crear un modelo completo desde el comienzo, debe crearse un modelo de alto nivel, el cual evolucionará a través del tiempo en forma incremental, o será posiblemente descartado.

Maximizar la inversión de las partes interesadas en el proyecto:

Sus usuarios invierten recursos (tiempo, dinero, facilidades, etc.) para tener un producto de software que satisfaga sus necesidades. Los usuarios merecen que sus recursos sean invertidos de la mejor manera posible y no malgastados por su equipo. Más aún, el usuario siempre tiene la última palabra sobre como deben invertirse o no sus recursos.

Modelar con un propósito:

Muchos desarrolladores se preocupan acerca de cuando sus artefactos (modelos, código fuente, documentos) son suficientemente detallados o demasiado detallados, o similarmente si son suficientemente exactos. Lo que normalmente no hacen es dar un paso atrás es preguntarse porqué están creando dicho artefacto y para quién. Con respecto al modelado, quizá necesite comprender mejor un aspecto del sistema, quizá necesite comunicar su enfoque a su jefe, o quizá necesite crear documentación que describa el sistema a las personas que se harán cargo de la operación y mantenimiento del mismo. ¿Si no puede identificar con claridad porque y para quién realiza un modelo, para que molestarse en hacerlo? El primer paso es identificar un propósito válido para el modelo y una audiencia para el mismo, luego basado en dicho propósito y audiencia debe desarrollarse el modelo con el suficiente nivel de detalle y precisión. Una vez que el modelo ha cumplido sus objetivos, se termina con él por el momento y puede dedicarse a otra actividad como escribir el código de dicho modelo para verlo en funcionamiento. Este principio también es aplicable a un cambio en un modelo: si se realiza un cambio a un modelo, por ejemplo para aplicar un patrón conocido, tiene que tener una razón válida para realizar dicho cambio (quizá para soportar un nuevo requerimiento o para refactorizar su trabajo de una manera más clara). Una implicación importante de este principio es que debe conocerse bien a la audiencia, aún cuando la audiencia sea uno mismo. Por ejemplo, si se esta creando un modelo para los desarrolladores de mantenimiento, que es lo que realmente ellos necesitan? ¿Necesitan un documento comprensivo de 500 páginas o es suficiente con un resumen de 10 páginas que explica el funcionamiento en general? Si se desconoce debe tratarse el tema con ellos.

Múltiples Modelos:

Potencialmente necesitará desarrollar múltiples modelos para desarrollar el software ya que cada modelo describe un aspecto simple del mismo. ¿Qué modelos son requeridos para modelar una

aplicación moderna? Considerando la complejidad del software actual, se necesita un amplio rango de técnicas en su caja de herramientas de modelado para ser efectivo. Un punto importante es que no necesita desarrollar la totalidad de dichos modelos para un sistema dado, pero dependiendo de la naturaleza del software que esté desarrollando, necesitará un subconjunto de los mismos. Diferentes sistemas, diferentes subconjuntos. Normalmente hay ciertos modelos que se utilizarán más a menudo que otros.

Trabajo de Calidad:

A nadie le agrada el trabajo mediocre. A la gente que lo realiza, no le gusta porque no se siente orgulloso del mismo. La gente que tratará con el mismo posteriormente, por ejemplo por mantenimiento, tampoco lo quiere porque es difícil de modificar. Y los usuarios finales tampoco lo quiere, ya que es un producto frágil que no cumple sus expectativas.

Rápida Retroalimentación:

El tiempo entre la acción y la retroalimentación sobre la acción es crítico. Trabajando con otra gente en un modelo, particularmente trabajando con una tecnología de modelado compartida (tales como pizarrones, tarjetas CRC, o elementos mas simples como notas autoadhesivas) se obtiene una retroalimentación casi instantánea sobre sus ideas. Trabajando estrechamente con sus clientes, para comprender sus requerimientos, para analizar dichos requerimientos, o para desarrollar la interfase de usuario que cubra sus expectativas, provee oportunidades de rápida retroalimentación.

El software es su objetivo primario:

El objetivo del desarrollo de software es producir software que cubra las necesidades del usuario en forma efectiva. El objetivo primario no es producir documentación extránea, extráneos artefactos de administración, o aún modelos. Cualquier actividad que no contribuya directamente a este objetivo debe ser cuestionada y evitada.

Viajar ligero de equipaje:

Cada artefacto que desarrolle y que decida conservar, necesitará mantenimiento en el tiempo. Si decide mantener siete modelos, siempre que ocurra un cambio (un nuevo o cambio de requerimientos, un nuevo enfoque a ser adoptado por su equipo, una nueva tecnología a ser adoptada, etc.), necesitará considerar el impacto de dichos cambios en los siete modelos y entonces actuar en concordancia. Si decide mantener solamente tres modelos, claramente tendrá menor trabajo para soportar el mismo cambio, haciéndolo más ágil por viajar más liviano. Similarmente, cuando más complejos y detallados sean sus modelos, más difícil y trabajoso será para aplicar cambios. Cada vez que se decide conservar un modelo, se realiza en compromiso entre agilidad y conveniencia de tener información disponible para posibilitar la comunicación entre su equipo y con el usuario. Nunca debe desestimarse la seriedad de este compromiso. Alguien que decida atravesar un desierto se beneficiará de un mapa, un sombrero, buenas botas, y una cantimplora con agua. Sin embargo no querrá hacerlo si debe cargar con cientos de galones de agua, un paquete completo de todos los elementos de supervivencia imaginables, y una colección de libros sobre el desierto. Similarmente, un equipo de desarrollo que decide desarrollar y mantener un documento de requerimientos detallado, una colección detallada de modelos del análisis, una colección detallada de modelos arquitecturales, y una colección detallada de modelos de diseño, pronto descubrirá que invierte la mayor parte de su tiempo manteniendo documentación en vez de escribiendo código.

Principios Adicionales:

El contenido es más importante que la representación:

Cualquier modelo dado puede tener distintas formas de representarse. Una implicación interesante es que el modelo no tiene porque ser un documento. Aún un complejo conjunto de diagramas creados con una herramienta CASE no necesita ser parte de un documento, en su lugar son utilizados como entradas para otros artefactos como el código fuente, pero nunca formalizados

como documentación oficial. El punto es que se toma ventaja del modelado sin incurrir en costos de creación y mantenimiento de documentación.

Todos pueden aprender de todos:

Uno puede nunca dominar un tema particular, siempre hay una oportunidad de aprender más y de extender su conocimiento. Tome la oportunidad de trabajar y aprender con otros, intente nuevas maneras de hacer las cosas. La tecnología cambia rápidamente. Tecnologías como Java evolucionan a un ritmo vertiginoso y nuevas tecnologías como C# y .NET son introducidas regularmente. Otras tecnologías no evolucionan tan rápidamente pero también lo hacen. El punto es que en la industria del software el cambio es la norma, y uno debe tomar cada oportunidad de aprender nuevas formas de hacer las cosas a través de cursos, educación, lecturas, y trabajando con otros.

Conozca sus modelos:

Ya que se tienen múltiples modelos que puede aplicar necesita conocer sus fortalezas y debilidades y la mejor forma de aplicarlos.

Conozca sus herramientas:

El software como herramientas de diagramación o herramientas de modelado, tiene una variedad de características. Si usted piensa utilizar una herramienta de modelado debe conocer sus características, y saber cuando utilizarlas.

Adaptaciones locales:

Su enfoque de desarrollo de software debe reflejar su entorno incluyendo la naturaleza de su organización, la naturaleza de sus usuarios, y la naturaleza del proyecto mismo. Los aspectos que pueden ser afectados incluyen: las técnicas de modelado que aplica (quizá sus usuarios insisten en interfaces de usuario concretas en lugar de simples bocetos o prototipos), las herramientas que utiliza (quizá deba utilizar una herramienta CASE ya adquirida por su organización), y el proceso de software que se sigue (su organización puede insistir en utilizar XP, RUP, o una metodología propia).

Comunicación abierta y honesta:

Las personas necesitan ser libres, y percibir que son libres para ofrecer sugerencias. Esto incluye ideas pertinentes a uno o más modelos, quizá alguien tiene una nueva forma de enfocar cierta porción del diseño, o una nueva visión de un requerimiento, la distribución de malas noticias como atrasos en el calendario, o simplemente el estado actual del trabajo. La comunicación abierta y honesta posibilita a la gente tomar mejores decisiones porque la información en la que se basan es mas exacta.

Trabaje con los instintos de las personas:

Cuando alguien siente que algo no va a funcionar, que algunas cosas son inconsistentes con otras, o que algo no "huele bien", es muy posible que así sea. A medida que usted gana experiencia desarrollando software, sus instintos se tornan más agudos, y lo que sus instintos le dictan subconscientemente puede ser a menudo una buena señal para sus esfuerzos de modelado. Si su instinto le dice que un requerimiento no tiene sentido, invéstiguelo con sus usuarios. Si su instinto le dice que una porción de la arquitectura no cubrirá las necesidades, realice un pequeño prototipo para probar su teoría. Si su instinto le dice que la alternativa de diseño A es mejor que la B, pero no tiene fundamentos, siga con la alternativa A por ahora. Es importante comprender que el valor del coraje le dice que usted podrá remediar alguna situación en el futuro si sus instintos fueron equivocados.

Prácticas del AM

AM define una colección de principios centrales y adicionales, basados en los principios de AM. Algunas de estas prácticas han sido adoptadas desde XP, pero son enfocadas en el esfuerzo de modelado.

Prácticas Centrales:

Participación activa de todos aquellos que soportan el proyecto:

Una expansión de la práctica cliente on-site de XP, que describe la necesidad de tener acceso on-site al usuario que tiene la autoridad y habilidad para proveer información pertinente al sistema que se está construyendo y tomar decisiones pertinente y en tiempo considerando los requerimientos y priorizándolos. AM expande la práctica cliente on-site de XP para involucrar directamente en el proyecto representantes del usuario, incluyendo usuario directos, sus administradores, gerentes, personal de operaciones y de soporte.

Aplicar los artefactos correctos:

Cada artefacto tiene su propia aplicación específica. Por ejemplo un diagrama de actividad de UML es útil para describir procesos de negocio, mientras que las estructuras estáticas de su base de datos son mejor representadas por un modelo físico de datos. Muy a menudo un diagrama es una opción mejor que el código fuente, porque se puede a menudo explorar alternativas de diseño más efectivamente dibujando un par de diagramas en un pizarrón junto con los compañeros de equipo, que escribiendo ejemplos de código. La implicancia es que debe conocerse las fortalezas y debilidades de cada artefacto para saber cuando aplicarlo y cuando no. Debe notarse que esto puede ser bastante dificultoso ya que se dispone de múltiples modelos.

Propiedad colectiva:

Todos pueden trabajar en cualquier modelo, y de hecho en cualquier artefacto del proyecto, si necesita hacerlo.

Considerar la prueba del sistema:

Cuando se modela debe preguntarse constantemente “¿como probaremos esto?” porque si no puede probarse el software no debería construirlo. Los procesos de software modernos incluyen las actividades de prueba y aseguramiento de la calidad a lo largo de todo el ciclo de vida del proyecto y muchos promueven escribir la prueba antes de escribir el software (esto es una práctica de XP).

Crear varios modelos en paralelo:

Debido a que cada tipo de modelo tiene sus fortalezas y debilidades un único modelo no es suficiente para cubrir sus necesidades de modelado. Por ejemplo cuando se está explorando requerimientos puede necesitar desarrollar algunos casos de uso esenciales, un prototipo de interfaz de usuario esencial, y algunas reglas de negocio. En combinación con la práctica de iterar hacia otro artefacto los modeladores ágiles a menudo descubrirán que es más productivo trabajar en varios modelos simultáneamente que enfocarse en uno solo a la vez.

Crear contenidos simples:

Usted debe mantener el contenido actual de sus modelos lo más simple posible mientras cubran las necesidades de sus clientes. La implicación es que no debe agregar aspectos adicionales a sus modelos a menos que se justifique. Por ejemplo, si no tiene el requerimiento de agregar características de auditoria a su sistema, no debería agregarlas en su modelo. Tenga el coraje de confiar que podrá agregar esas características cuando sean requeridas. Esto está acorde a la práctica de XP de Diseño Simple.

Representar los modelos con simplicidad:

Cuando considera los diagramas potenciales que puede aplicar (diagramas de UML, diagramas de interfaz de usuario, modelos de datos, etc.) comprende rápidamente que la mayor parte del tiempo solo necesita un subconjunto de todos estos modelos. Un modelo simple que muestre las características clave que intenta comprender, quizá un modelo de clases representando las responsabilidades primarias de las clases y las relaciones entre ellas, a menudo prueba ser suficiente.

Presentar los modelos públicamente:

Debe mostrar sus modelos públicamente, a menudo sobre algo llamado “el panel de modelado” o el “muro de los milagros”. Esto soporta la *comunicación abierta y honesta* en el equipo ya que todos los modelos son fácilmente accesibles por todos, como también por los usuarios. Su muro de modelado es donde usted pone sus modelos para que cualquiera los pueda ver. Su muro de modelado puede ser físicamente una pizarra o estar implementado como un sitio Web.

Iterar a otros artefactos:

Cuando trabaja en un artefacto de desarrollo (un caso de uso, un tarjeta CRC, un diagrama de interacción, etc.), y se da cuenta que se encuentra atascado en el mismo, debe considerar comenzar a trabajar con otro artefacto. Cada artefacto tiene sus fortalezas y debilidades. Cada artefacto es adecuado para un tipo de trabajo. Siempre que encuentre dificultades trabajando con un artefacto, es una señal de que debe considerar trabajar con otro artefacto. Por ejemplo si se encuentra trabajando sobre un caso de uso esencial, puede considerar el comenzar a trabajar con un prototipo esencial de UI, un modelo CRC, una regla de negocio, etc. Iterando hacia otro artefacto, se sale del atascamiento porque uno sigue progresando sobre otro artefacto. Más aún, cambiando su punto de vista a menudo describe que su enfoque anterior era lo que lo mantenía atascado.

Modelar en pequeños incrementos:

El desarrollo incremental en el cual organiza un gran esfuerzo en porciones menores que libera a través del tiempo, en incrementos de unas pocas semanas o un par de meses, incrementa su agilidad posibilitando entregar software funcionando en forma más rápida.

Modelar con otros:

A menudo encontrará que su equipo de desarrollo necesita trabajar conjuntamente para crear un conjunto de modelos centrales críticos para su proyecto. Por ejemplo para desarrollar la arquitectura base del sistema, deberá trabajar con un grupo de gente para desarrollar una solución con la que todos estén de acuerdo, como también la solución mas simple.

Pruébelo con código:

Un modelo es una abstracción que debe reflejar con precisión un aspecto del sistema en construcción, pero para determinar si el mismo funcionará debe probarse con código. Por ejemplo, si se encuentra desarrollando un diagrama de secuencia que modela un proceso de negocio, escriba el código de prueba, el código de los objetos del negocio, y asegúrese de que funciona correctamente.

Use las herramientas más simples:

La gran mayoría de los modelos pueden representarse sobre un pizarrón o sobre un papel. Siempre que se desee almacenar uno de dichos diagramas se puede tomar una fotografía del mismo, o simplemente transcribirlo en papel. El verdadero valor de un diagrama radica en dibujarlo para pensar sobre un problema, y una vez que este es resuelto, el diagrama no ofrece ya mucho valor. Como resultado, una pizarra y un marcador suelen ser a menudo la mejor alternativa. Debe utilizarse una herramienta de dibujo para crear un diagrama a presentar a representantes importantes del proyecto, y ocasionalmente, puede utilizarse una herramienta de dibujo si y solo si provee algún valor a mis esfuerzos de programación, como la generación de código.

Prácticas Adicionales:

Aplique estándares de modelado:

Esta práctica está renombrada de la práctica Codificación Estándar de XP. La idea básica es que el desarrollador siga un conjunto de estándares de modelado en un proyecto de software. Así como es valioso la aplicación de convenciones comunes en la codificación, para obtener un código mas sencillo de entender y evolucionar, ocurre lo mismo en el seguimiento de convenciones de modelado comunes. Hay una amplia variedad de estándares de modelado comunes disponibles, incluyendo UML que define una notación y semántica para modelos orientados a objetos comunes. UML provee un buen conjunto de técnicas pero no es suficiente. No todos los posibles artefactos de modelado son incluidos en UML. Más aún, UML tampoco dice nada acerca de estilos de modelado. Un estándar puede ser por ejemplo, representar las clases de objetos con rectángulos, en tango un estilo puede implicar que las jerarquías de herencia se representen siempre verticalmente.

Aplicar patrones cuidadosamente:

Modeladores efectivos aprenden y luego aplican patrones arquitecturales, de diseño, y de análisis en sus modelos. Sin embargo, como señala Martin Fowler en [Is Design Dead?](#), los desarrolladores deben considerar la simplicidad en la aplicación de un patrón, para aplicarlo cuidadosamente. Esto refleja el valor de simplicidad. Por ejemplo, si se aplica un patrón hoy del cual se utilizará una mínima funcionalidad, pero se espera que en el futuro sea sencillo refactorizar para extender al funcionalidad, es una buena aplicación. En otras palabras no debe sobre-modelarse.

Descartar modelos temporales:

La gran mayoría de modelos que se crean son modelos temporarios de trabajo, que una vez que cumplen su propósito, no agregan mucho valor. Los modelos rápidamente pierden sincronismo con el código, y no hay nada malo en ello. Debe tomarse entonces la decisión de sincronizar los modelos si agregan valor o simplemente descartarlos.

Formalice Modelos Contractuales:

Modelos de contrato son a menudo requeridos cuando un grupo externo controla una fuente de información que su sistema requiere, como ser una base de datos, una aplicación legacy, o un servicio de información. Un modelo de contrato es algo que ambas partes deben acordar mutuamente y mutuamente modificar cuando sea requerido. Ejemplos de modelos de contrato incluyen la documentación detallada de una API, la descripción del diseño de registro de un archivo, etc. Un modelo de contrato, a menudo requiere la utilización significativa de recursos en su desarrollo y mantenimiento. El objetivo es minimizar los modelos de contrato para cumplir con el principio de *viajar liviano*. Normalmente se utilizan medios electrónicos para crear un modelo de contrato, ya que el mismo deberá ser mantenido en el tiempo.

Modelar para comunicar:

Una objetivo secundario del modelado es para comunicarse con personas externas al equipo o para crear modelos de contrato. Debido a que los clientes para estos modelos están fuera del equipo, debe invertirse tiempo en hacer modelos “prolijos” utilizando normalmente herramientas electrónicas como diagramadores o procesadores de palabras.

Modelar para comprender:

La aplicación más importante del modelado es para comprender el espacio del problema, para identificar y analizar requerimientos para el sistema, o para comparar alternativas de diseño potenciales, para identificar potencialmente la solución mas sencilla que cubra los requerimientos.

Reutilizar recursos existentes:

Existe una abundancia de información de la que un modelador ágil puede tomar ventaja. Por ejemplo algunos patrones de análisis y diseño pueden ser útiles para su sistema. O quizá pueda

tomarse ventaja de modelos de requerimientos empresariales existentes, modelos de procesos de negocio, modelos físicos de datos, etc.

Actualizar solo cuando sea perjudicial:

Debe actualizar un modelo solo cuando, el tener el modelo desactualizado produzca mayor perjuicio que el costo de actualizarlo. Con este enfoque descubrirá que solo actualiza un número pequeño de modelos, ya que los modelos no necesitan ser perfectos para aportar valor. Un mapa por ejemplo, puede estar desactualizado pero puede aún seguir siendo útil.

Buenas Ideas:

Las siguientes prácticas son complementarias a AM, pero no están incluidas como parte de el:

Refactorización:

Esta es una práctica de codificación en la cual se realizan pequeños cambios, llamados refactorizaciones, para que el código soporte nuevos requerimientos o para mantener el código lo más simple posible. Desde el punto de vista de AM esta práctica asegura que su diseño permanezca claro y limpio a menudo que se trabaja en el código. Refactorización es una parte integral de XP.

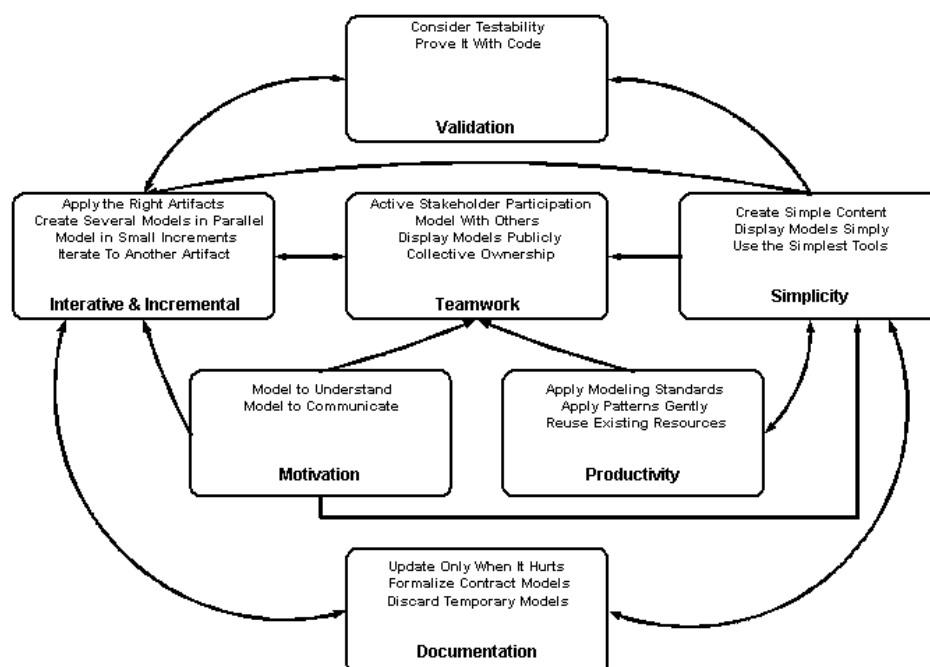
Prueba antes del diseño:

Esta es una práctica del desarrollo donde primero se considera los casos de prueba antes de escribir el código. Desde el punto de vista de AM esta práctica fuerza a pensar a través del diseño antes de escribir el código, removiendo la necesidad de modelado detallado del diseño. Prueba antes del diseño es una parte integral de XP.

Integración entre las prácticas de AM

Las prácticas de AM son sinérgicas, ya que se soportan mutuamente y a veces unas posibilitan otras.

La siguiente figura muestra la relación entre las diferentes prácticas de AM organizadas en siete categorías:



Las primeras cuatro categorías – Validación, Iterativo e Incremental, Equipo de trabajo, y Simplicidad – consolidan las prácticas centrales de AM, las principales a adoptar para ser realmente modelado ágil. Las prácticas adicionales se agrupan en las categorías Documentación, Motivación y Productividad.

Integración de prácticas centrales:

Categoría “Trabajo en Equipo”:

Está integrada por cuatro prácticas – Participación Activa del Usuario, Modelar con Otros, Desplegar Públicamente los Modelos, y Propiedad Colectiva.

La Participación Activa del Usuario es crítica ya que es para él para quién se desarrolla el sistema y cuyos requerimientos deben ser comprendidos y satisfechos. Esta práctica es a su vez soportada por la práctica de Modelar con Otros, siendo el usuario en este caso el “otro”.

Similantemente la participación activa no solo del usuario, sino de todos los miembros del equipo, posibilita tener una visión más completa del problema a modelar, ya que aspectos que pasan desapercibidos para algunos pueden ser detectados por otros. La práctica de Propiedad Colectiva extiende el trabajo en equipo ya que cuando una sola persona “posee” un modelo, se convierte en un cuello de botella en el equipo. La práctica de Desplegar Públicamente los Modelos facilita poder ver hacia delante y atrás entre los modelos considerando la información contenida en los modelos a la vez.

Categoría “Iterativo e Incremental”:

Está integrada por cuatro prácticas – Aplicar los Artefactos Correctos, Crear Varios Modelos en Paralelo, Iterar Hacia Otro Artefacto, y Modelar en Pequeños Incrementos.

Cada artefacto de modelado tiene sus fortalezas y debilidades. Por otra parte, no existe un único modelo que cubra todos los aspectos del sistema que se está modelando. Así mismo, es muy improbable que el usuario plantee sus requerimientos en forma ordenada de forma tal que el modelador pueda concentrarse primero en el modelado de casos de uso, luego pasar al modelado de reglas de negocio, luego al modelado de requerimientos no funcionales, y así. Por el contrario, por lo general el usuario plantea sus requerimientos en forma desordenada, y recurrentemente vuelve hacia atrás ampliando detalles de requerimientos ya planteados. Esto hace que la captura de requerimientos sea una práctica altamente dinámica y que requiera la construcción de varios modelos concurrentemente. Lo mismo ocurre con las actividades de análisis, arquitectura, y diseño.

La práctica Crear Varios Modelos en Paralelo permite capturar un amplio rango de información. Esta práctica es claramente soportada por las prácticas de Aplicar los Artefactos Correctos e Iterar Hacia Otro Artefacto. Esta última a su vez es soportada por Modelar en Pequeños Incrementos. Típicamente se modela un poco en un artefacto, luego en otro, luego en otro, y así.

Categoría “Simplicidad”:

Está integrada por tres prácticas – Crear Contenidos Simples, Representar los Modelos Simplemente, Utilizar las Herramientas más Sencillas -. Las prácticas Crear Contenidos Simples y Representar los Modelos con Simplicidad, claramente apunta a la simplicidad del modelo. Utilizando las Herramientas más Sencillas se incrementan las posibilidades de representar simplemente estos modelos.

Categoría “Validación”:

Está integrada por dos prácticas centrales – Considerar la Prueba, y Probar con Código-. Una filosofía útil es que todo lo que vaya a construirse debe poder ser probado, sino no hay que construirlo. Debe considerarse la prueba de cada artefacto que se construye. Probando un artefacto con código, se comprueba que el sistema es verificable.

Integración de Prácticas Adicionales:

Categoría “Documentación”:

Está integrada por tres prácticas adicionales – Descartar los Modelos Temporales, Formalizar Modelos de Contrato, Actualizar Solo Cuando sea Dañino. A medida que el desarrollo de un proyecto evoluciona, muchos de sus artefactos cambian. Los modelos deben ser actualizados para reflejar dichos cambios. Una práctica acosenjada para adoptar un enfoque ágil es actualizar los modelos solo cuando sea dañino o peligroso. De esta forma cuando se encuentran modelos que no se actualizan desde tiempo, puede ser un indicativo de que dichos modelos pueden ser transitorios y descartables. Una excepción la constituyen los modelos utilizados para formalizar contratos con sistemas externos. Dichos modelos rara vez cambian, sin embargo deben ser conservados.

Categoría “Motivación”:

Está integrada por dos prácticas adicionales – Modelar Para Comunicar, y Modelar Para Comprender. Estas prácticas no están muy relacionadas entre si, pero entre ambas dan soporte y posibilitan la aplicación de las demás prácticas.

Categoría “Productividad”:

Está integrada por tres prácticas adicionales – Aplicar Estándares de Modelado, y Aplicar Patrones Cuidadosamente, y Reutilizar Recursos Existentes. La práctica de Reutilizar Recursos Existentes, en cierta sentido una predisposición a tomar beneficio del trabajo de otros. Esta predisposición promueve la utilización de patrones, que es una manera productiva de reutilizar el trabajo realizado por otros y de probada eficacia. Esta predisposición también promueve el seguimiento de prácticas estandarizadas.

Desarrollo Ágil de Software

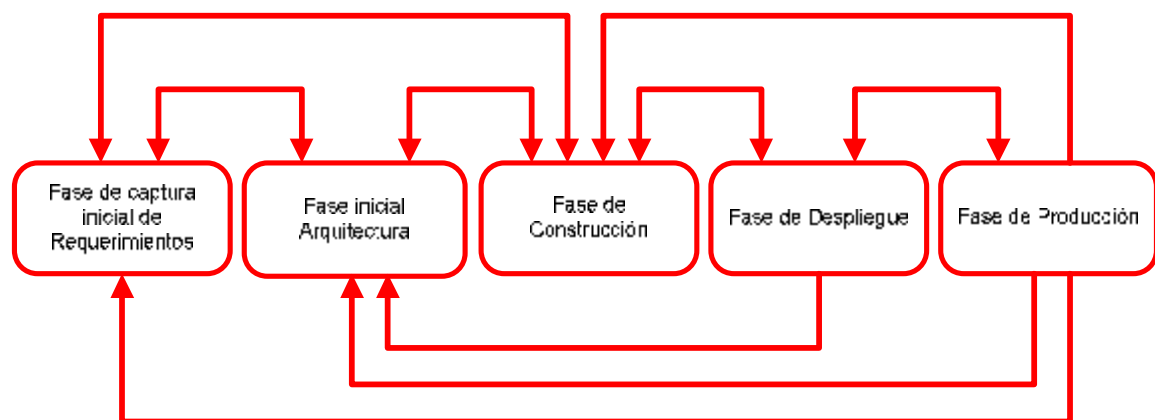
Ciclo de Vida para el Desarrollo Ágil de Software

El propósito de AM es definir prácticas de modelado y documentación efectivas que deben adaptarse en otro proceso más abarcativo como ser XP o RUP.

Un proceso de desarrollo de software ágil se caracteriza por ser secuencial a largo plazo, iterativo en el corto plazo, y entregar versiones incrementales en el tiempo.

Secuencial a largo plazo no significa que el proceso sea en cascada como los enfoques tradicionales, significa que a lo largo del ciclo de vida el enfoque se va desplazando desde la captura inicial de requerimientos de alto nivel, a la determinación de la arquitectura, construcción, despliegue, y finalmente actividades de producción (Ej. operaciones y soporte).

La siguiente figura representa un ciclo de vida de alto nivel para el desarrollo ágil de software:



Las fases indicadas en esta figura son arbitrarias y son introducidas para describir un proceso generalizado. El proceso que se decida adoptar y adaptar a las prácticas de AM puede tener un conjunto de actividades prescriptas distintas.

Algo importante a destacar es que cuando se dice secuencial a largo plazo no significa que las actividades se realicen estrictamente en orden secuencial. Los flujos que retornan a actividades previas representan precisamente que se vuelven a realizar dichas actividades en sucesivas iteraciones. El concepto de secuencial a largo plazo significa que el enfoque va cambiando a medida que se avanza en el ciclo de vida del proyecto.

Iterativo en el corto plazo es un enfoque que proviene del desarrollo en cascada propuesto originalmente por Bohem, y significa que se realiza un poco de requerimientos, modelado, codificación, prueba, y luego nuevamente se vuelve hacia atrás, avanzando de esta forma de a pequeños incrementos.

El desarrollo incremental se basa en el concepto de que una aplicación puede ser suministrada en múltiples entregas, entregas que a menudo son construidas sobre funcionalidades de entregas

anteriores. De hecho la gran mayoría de los sistemas son desarrollados siguiendo este enfoque, se entrega una versión inicial del sistema cuya funcionalidad es extendida en el tiempo.

Integración de AM con XP (Programación Extrema)

XP es una metodología con un alcance mayor que AM, en la cual dichas prácticas pueden ser adaptadas. Más aún, si bien XP incluye el modelado como parte de su proceso no es muy explícito acerca de como debe ser realizado. He aquí una oportunidad para AM. Así mismo, debido a que tanto XP como AM son ambos métodos ágiles, es mucho más sencilla su integración que en el caso de metodologías prescriptivas como RUP.

Existen varias apreciaciones incorrectas acerca de XP y el modelado. Las tres principales son que en un proyecto XP no se modela, en un proyecto XP no se documenta, o si se modela, los únicos artefactos disponibles son los de UML.

Más allá de la integración existente entre las prácticas AM y XP, existe una correspondencia filosófica entre ambos métodos.

Aplicación de prácticas AM y prácticas XP

Práctica AM	XP
Participación activa del Stakeholder	Esta práctica se corresponde con la práctica Cliente en el Lugar, de XP. AM utiliza el término soporte del proyecto (stakeholder), y enfatiza en participación activa, no solo en su presencia.
Aplicar estándares de modelado	Esta práctica se corresponde directamente con la práctica Codificación Estándar de XP.
Aplicar patrones cuidadosamente	Esta práctica se condice con el principio YAGNI de XP y está en concordancia con la práctica de diseño simple.
Aplicar los artefactos correctos	Esta práctica no es explícita en XP, aunque se corresponde con la filosofía XP "si lo necesita, hágalo" y de utilizar las herramientas más correctas.
Propiedad colectiva	Esta es una práctica original de XP.
Considerar la prueba	Esta práctica es un reflejo de la práctica XP de Probar antes de codificar.
Crear varios modelos en paralelo	Esta es una práctica específica del modelado que los desarrolladores XP pueden adoptar si lo desean.
Crear contenido simple	Esta es una práctica complementaria a la práctica XP Diseño Simple.
Representar modelos simplemente	Ídem al punto anterior.
Descartar modelos temporarios	Esta práctica refleja el principio de XP de Viajar Liviano que AM ha adoptado.
Presentar los modelos públicamente	Esta práctica refleja el valor XP de <i>Comunicación</i> , el principio XP de <i>Comunicación Abierta y Honesta</i> , y refleja la práctica de <i>Propiedad Colectiva</i> .
Formalizar modelos contractuales	Esta práctica no tiene correspondencia directa en XP, sin embargo se corresponde con la filosofía XP "si lo necesita, hágalo".
Iterar hacia otro artefacto	Esta práctica se corresponde con la práctica de los desarrolladores XP de iterar entre diferentes artefactos de modelado como ser código, tarjetas CRC, y pruebas.
Modelar en pequeños	Esta práctica soporta el enfoque XP de desarrollar iterativamente en pequeños incrementos. Tanto AM como XP prefieren un enfoque incremental

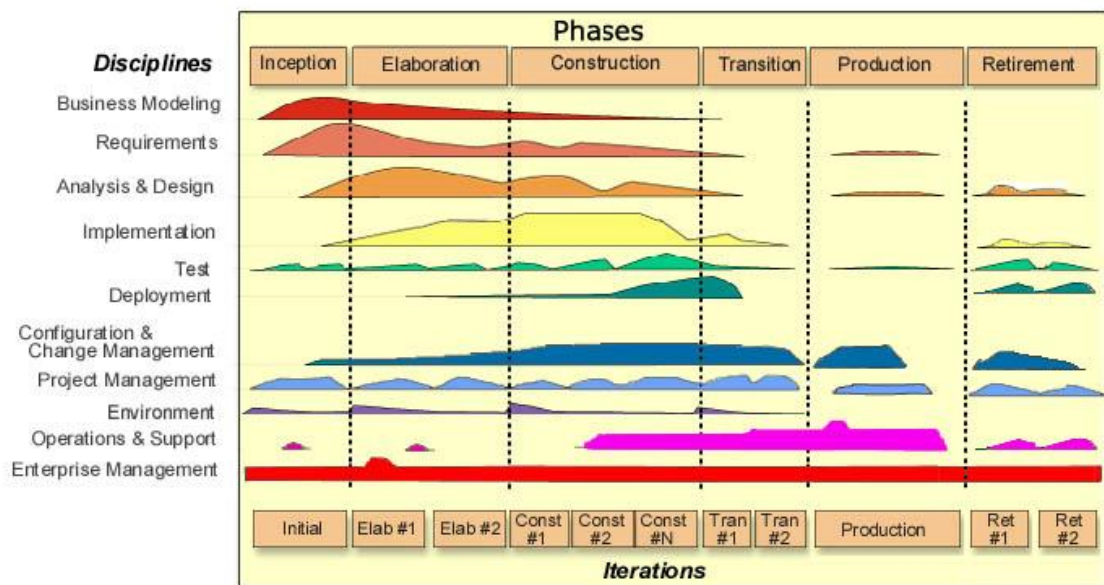
incrementos		en vez del enfoque un gran diseño frontal (big design up front BDUF).
Modelar para comunicar		Esta es una práctica específica de modelado, que refleja el principio XP de <i>Comunicación Abierta y Honesta</i> .
Modelar para comprender		Esta es una práctica específica del modelado, que describe la razón primaria por la cual modelamos. Es consistente con el uso de tarjetas CRC en XP.
Modelar con otros.		Esta práctica se corresponde con la práctica XP de programación de a pares.
Probar código	con	Esta es la versión AM del principio XP de realizar Experimentos Concretos.
Reutilizar recursos existentes		Si bien no es un concepto explícitamente incorporado en XP, tampoco esta explícitamente excluido.
Actualizar solo cuando sea peligroso.	solo sea	Esta práctica de AM refleja el principio XP de Viajar Liviano.
Utilizar las herramientas más sencillas	las	Esta práctica refleja el principio XP de Asumir Simplicidad, y es consistente con la preferencia XP de herramientas sencillas como ser las tarjetas CRC .

Integración de AM con RUP (Proceso Unificado de Rational)

Como funciona el modelado en el Proceso Unificado de Rational (RUP)

En RUP, todos los esfuerzos, incluyendo el modelado, se organizan en disciplinas (llamadas formalmente flujos de trabajo), y son realizadas en un modo iterativo e incremental.

El siguiente diagrama representa el ciclo de vida del proceso unificado:



Claramente RUP es una metodología secuencial a largo plazo e iterativa en el corto plazo. Las fases en RUP ocurren secuencialmente. Estas fases son descompuestas en iteraciones. Dentro de

cada iteración se realiza un poco de cada una de las disciplinas centrales: *Modelado del Negocio, Requerimientos, Análisis, Diseño, Implementación, y Prueba.*

Modelado del Negocio: El propósito de esta disciplina es modelar el contexto del negocio y el alcance del sistema.

Requerimientos: El propósito de esta disciplina es realizar la ingeniería de requerimientos del proyecto, incluyendo la identificación, modelado, y documentación de dichos requerimientos. El principal entregable de esta disciplina es la especificación de requerimientos del Software (SRS).

Análisis y Diseño: El propósito de esta disciplina es desarrollar una arquitectura robusta para el sistema basada en los requerimientos, transformar dichos requerimientos en un diseño, y asegurar que el entorno de implementación se refleje en su diseño.

Algo importante a comprender es que para que la aplicación de AM sea exitosa, la cultura de la organización debe abrirse a sus valores, principios, y prácticas. El problema es que a menudo RUP es adoptado por organizaciones que implícita o explícitamente no aceptan los valores del modelado ágil. Su enfoque es a menudo seguir procesos y utilizar herramientas (como el producto RUP). Claramente esto va en contra de AM que prefiere individuos e interacciones a procesos y herramientas.

Práctica AM	RUP
Participación activa del Stakeholder	AM tiene una definición amplia de stakeholders del pro. RUP incluye participación activa stakeholders en todas sus disciplinas. Para ser exitosos los equipos de proyecto RUP deben incluir stakeholders en roles de modelado.
Aplicar estándares de modelado	La aplicación de estándares de modelado, en particular diagramas UML, es parte integral de RUP. Más aún el producto RUP incluye guías para la construcción de artefactos de modelado. Sin embargo para permanecer ágiles los equipos RUP deben reconocer cuando flexibilizar dichas guías.
Aplicar patrones cuidadosamente	Los equipos RUP son libres de aplicar patrones de diseño. El producto RUP describe varios patrones de diseño que se pueden aplicar durante el modelado. Sin embargo no especifica que dicha aplicación debe realizarse con sencillez y cuidado.
Aplicar los artefactos correctos	Una de las fortalezas de RUP es que provee consejos sobre cuando aplicar cada tipo de modelo. Algunas versiones modernas del producto RUP incluyen guías de aplicación de artefactos no UML.
Propiedad colectiva	El concepto de propiedad colectiva de AM puede utilizarse para extender las prácticas de RUP, asumiendo que la cultura del equipo soporta el concepto de comunicación abierta y honesta. RUP soporta el concepto de propiedad colectiva con un fuerte énfasis en la administración de configuraciones.
Considerar la prueba	RUP incluye explícitamente la prueba en su ciclo de vida. RUP también incluye muchas posibilidades para revisar artefactos de modelado.
Crear varios modelos en paralelo	RUP incluye explícitamente este concepto.
Crear contenido simple	Esta práctica es una opción realizada por el modelador aunque debe ser implícitamente soportada por el equipo. Los equipo RUP necesitan adoptar guías de modelado que permitan crear modelos solo lo suficientemente buenos para el equipo y el cliente. Esta es una cuestión mas cultural a menudo difícil de adoptar por la organización.
Representar modelos simplemente	Ver punto anterior
Descartar modelos temporarios	Los modeladores de equipos RUP tienen la libertad de descartarlos cuando deseen. Al igual que con la simpleza en los modelos, esto es más una

	cultura organizacional.
Presentar los modelos públicamente	Los equipos RUP son libres de seguir esta práctica.
Formalizar modelos contractuales	RUP incluye el concepto de integración con sistemas externos. Estos sistemas son típicamente identificados en los modelos de casos de uso y RUP sugiere la introducción de clases de frontera para implementar las interfaces a estos sistemas.
Iterar hacia otro artefacto	Esta práctica puede ser fácilmente adoptada por un equipo RUP. La representación casi secuencial de actividades de modelado y la división de estas en disciplinas separadas, dificulta la actitud iterativa necesaria para los modeladores ágiles.
Modelar en pequeños incrementos	Esta práctica es claramente un aspecto de RUP. El soporte de RUP para iteraciones implica que el modelo se desarrolla incrementalmente a lo largo del proyecto.
Modelar para comunicar	RUP implícitamente incluye esta práctica. Los equipos RUP pueden comunicar de diversa forma siguiendo el principio de modelar con un propósito conociendo cual es su audiencia y cuales son los requerimientos.
Modelar para comprender	Ver punto anterior
Modelar con otros.	RUP implícitamente incluye esta práctica. Cada disciplina de modelado claramente incluye varios roles, cada rol ocupado por una o más personas.
Reutilizar recursos existentes	Reutilización es una parte implícita de RUP
Actualizar solo cuando sea peligroso.	En teoría este es un concepto sencillo de adoptar para los equipos RUP ya que dramáticamente reduce el esfuerzo de actualizar los artefactos. Sin embargo en la práctica es difícil de adoptar en algunas organizaciones sobre todo si tienen un fuerte énfasis en la "trazabilidad".
Utilizar las herramientas más sencillas	El producto RUP incluye tutores sobre las herramientas que facilitan el trabajo con herramientas provistas por Rational. Así también los equipos RUP pueden extender su productividad haciendo uso de herramientas simples como pizarras, CRC cards, etc.

Adoptando AM en un proyecto RUP

Para que un equipo RUP adopte las prácticas de AM debe reconsiderar las siguientes cuestiones:

1. **Olvidar el término "conducido por casos de uso".** Los casos de uso son una buena técnica para documentar requerimientos de comportamiento pero esta es solo una parte de los requerimientos funcionales, y más aún una pequeña parte de la totalidad de los requerimientos. Por este motivo es que RUP incorpora un documento llamado de especificación suplementaria. Los requerimientos conducen las cosas. Los casos de uso no.
2. **Reconocer que existen más artefactos de modelado que los descriptos por UML.** Existen artefactos no especificados por UML a disposición del modelador. Inclusive recientes versiones de UP incluyen soporte para modelos de datos e interfaces de usuarios.
3. **Reconocer que RUP no es inherentemente centrado en la documentación.** RUP es actualmente muy claro en que solo deben desarrollarse los documentos actualmente necesarios. RUP incluye tres mayores conjuntos de artefactos orientados al modelado – modelado del negocio, modelado de requerimientos, análisis y diseño. Cada grupo posee un conjunto de artefactos. Es muy probable que en un proyecto no se requieran todos los artefactos. Si un artefacto no es necesario, tampoco es necesario conservarlo como documentación.

4. **Construir una visión común entre los desarrolladores y soportes del proyecto.** Los administradores a menudo tienden a procesos prescriptivos, algo bien definido y comprensivo como RUP. Los desarrolladores por otra parte, prefieren técnicas ágiles como XP y AM. Afortunadamente RUP es lo suficientemente flexible como para adaptarse a un enfoque razonablemente ágil, pero esto bajo un consenso entre administradores y desarrolladores.
5. **Promover activamente el desarrollo iterativo e incremental.** Tradicionalmente los desarrolladores tienden a adoptar un estilo de desarrollo caracterizado por un gran trabajo inicial de requerimientos y la focalización en artefactos específicos. Debe cambiarse este enfoque hacia el modelado con múltiples artefactos utilizándolos en iteraciones breves. Cada iteración debe producir un incremento en el producto en desarrollo. Cada iteración debe entregar un producto operativo con incrementos en su funcionalidad.
6. **Promover activamente la simplicidad.** Muchos modeladores experimentados tienen tendencia a especificar modelos con muchos más detalles de los que se necesitan actualmente. Por ejemplo algunos modeladores esbozan como sería el código de sus modelos, cuando solo deben enfocarse en la creación de un modelo de clases mínimo que alcance los objetivos. Manteniendo los modelos simples se trabaja más rápido creando prontamente algo de valor para los programadores.
7. **Formar el equipo de proyecto con generalistas competentes.** Muchas organizaciones tienen posiciones diferentes para sus modeladores, poniendo foco en las especialidades, una práctica que reduce la agilidad del equipo. Aunque RUP claramente pone de manifiesto que una misma persona puede ocupar diferentes roles, por lo general esto no es tenido en cuenta en las organizaciones que implementan RUP.
8. **Viva el principio de Comunicación Abierta y Honesta de AM.** La práctica de desplegar públicamente los modelos muchas veces es rechazada principalmente por temor de las personas del uso que puedan hacer otros de los mismos. Normalmente cuando se supera estas trabas, se descubren los beneficios de la comunicación abierta y honesta.

Una Introducción a XP

¿Qué es la programación Extrema?

Programación extrema es una disciplina de desarrollo de software basada en valores de simplicidad, comunicación, retroalimentación, y coraje. Se basa en la utilización de prácticas simples, con suficiente retroalimentación para posibilitar al equipo ver la situación en que se encuentra y afinar sus prácticas a su situación específica.

XP fue creada en respuesta a dominios de problema muy dinámicos con alto y rápido cambio de requerimientos.

XP está orientado a pequeños equipos de trabajo. Típicamente entre 2 a 10 personas. Con equipos más grandes no puede o es de difícil aplicación.

XP requiere también un equipo de trabajo extendido que incluya no solo desarrolladores sino también administradores, clientes, y todas las personas de alguna u otra forma involucradas con el proyecto (stakeholders).

Un enfoque importante de XP es la prueba. Las pruebas unitarias y de integración deben automatizarse.

XP es una metodología "liviana". Tiene pocas reglas y un modesto número de prácticas sencillas de seguir.

En Programación Extrema, cada involucrado (stakeholder) en el proyecto es una parte integral del equipo de trabajo. El equipo de trabajo se conforma alrededor de un representante del negocio denominado "el Cliente", quién se sienta con el equipo y trabaja con el diariamente.

- **Práctica Central: Equipo Total**

Los equipos de Programación Extrema utilizan una forma simple de planificación y seguimiento para decidir que debe hacerse a continuación, y para predecir cuando se concluirá el proyecto. Enfocándose en los valores del negocio, el equipo produce el software en una serie de pequeñas entregas completamente integradas que superan todas las pruebas definidas por el Cliente.

- **Prácticas Centrales: Juego de Planificación, Pequeñas Entregas, Pruebas del Cliente**

Los Programadores Extremos trabajan juntos de a pares y como un grupo, con diseños simples y código obsesivamente probado, mejorando el diseño continuamente para mantenerlo siempre acorde a las necesidades actuales.

- **Prácticas Centrales: Diseño Simple, Programación de pares, Desarrollo Prueba-primero (Test-First Development), Mejora del Diseño**

El equipo de Programación Extrema mantiene el sistema integrado y funcionando todo el tiempo. Los programadores escriben todo el código de producción de a pares, y todos trabajan juntos todo el tiempo. Codifican en un estilo consistente de forma tal que cualquiera pueda comprender y mejorar el código como sea necesario.

- **Prácticas Centrales: Integración Continua, Propiedad Colectiva del Código, Codificación Estándar**

El equipo de Programación Extrema comparte una visión común y simple de cómo debe verse el sistema. Cada uno trabaja a un paso que puede ser sostenido indefinidamente.

- **Prácticas Centrales: Metáfora, Paso sostenido**

Prácticas Centrales de XP

Equipo Total

Todos los stakeholders de un proyecto XP se sientan juntos, miembros de un equipo. Este equipo debe incluir un representante del negocio – “el Cliente” – quién provee los requerimientos, prioridades, y conduce el proyecto. Es mejor si el Cliente o uno de sus asistentes es un usuario final real que conoce el dominio y que se necesita.

El equipo ha de tener por supuesto programadores.

El equipo debe incluir personal de pruebas, que ayudarán al Cliente a definir las pruebas de aceptación.

Los analistas ayudarán al Cliente a definir los requerimientos.

Existe comúnmente un líder (coach) que ayuda al equipo a mantener la dirección, y facilita el proceso.

Puede haber un administrador suministrando recursos, manejando la comunicación externa, y coordinando actividades.

Ninguno de estos roles es necesariamente propiedad exclusiva de un individuo. Cualquiera en un equipo XP contribuye en cualquier forma. Los mejores equipos no tienen especialistas, solo contribuyentes generales con habilidades especiales.

Juego de Planificación

XP aborda dos cuestiones claves en el desarrollo de software: predecir que será realizado para la fecha de vencimiento, y determinar que hacer a continuación. El énfasis está en la conducción del proyecto más que en la exacta predicción de qué será necesario y cuanto tiempo llevará, lo cual es bastante difícil de establecer.

Hay dos niveles de planificación claves en XP, abordando estas dos cuestiones:

Planificación de Entregas (Release Planning) es una práctica donde el Cliente presenta las características deseadas, y los programadores estiman su dificultad. Con la estimación de costo en mano, y con el conocimiento de la importancia de las características, el Cliente diseña el plan para el proyecto. El plan de entrega inicial es impreciso. Ni las prioridades, ni las estimaciones son sólidamente verdaderas, y hasta que el equipo comienza a trabajar, no sabremos cuán rápido realmente se avanzará.

Este primer plan de entrega apenas tiene la precisión suficiente para tomar decisiones, de cualquier modo, un equipo XP revisa y actualiza el plan de entregas regularmente.

Planificación de Iteraciones (Iteration Planning) es la práctica por la cual el equipo recibe dirección cada par de semanas. Los equipos XP construyen software en “iteraciones” de dos semanas, entregando software ejecutable útil al final de cada iteración. Durante la Planificación de Iteraciones, el Cliente presenta las características deseadas para las próximas dos semanas. Los programadores las descomponen en tareas, y estiman su costo (en un nivel de detalle mas fino que en la planificación de entregas). Basados en la cantidad de trabajo realizado en la iteración previa, el equipo estima cuanto trabajo será necesario en la siguiente iteración

Estos pasos de planificación son muy sencillos, sin embargo proveen muy buena información y excelente control del proyecto en manos del Cliente. Cada par de semanas, la cantidad de progreso es enteramente visible. No existe el síndrome del 95% en XP. Una historia de características esta completa o no lo está.

Pruebas del Cliente

Como parte de la presentación de cada característica deseada, el Cliente define una o más pruebas automatizadas de aceptación para determinar si la característica funciona. El equipo utiliza estas pruebas para demostrarse a si mismo y al Cliente que la característica está implementada

correctamente. La automatización de la prueba es importante, porque bajo la presión del tiempo, las pruebas manuales son dejadas de lado.

El equipo XP trata las pruebas del Cliente de la misma forma que las pruebas de los programadores: una vez que la prueba corre aceptablemente, el equipo la debe continuar ejecutándola correctamente a posteriori.

Pequeñas Entregas

Los equipos XP entregan software ejecutable probado, de valor para el negocio elegido por el cliente, al final de cada iteración. El Cliente puede utilizar este software para cualquier propósito, ya sea de evaluación o aún liberarlo a los usuarios finales (recomendado). El aspecto más importante es que el software es visible, y se entrega al Cliente al final de cada iteración.

Diseño Simple

Los equipos XP desarrollan software de diseño simple. Un equipo XP mantiene el diseño para soportar específicamente la funcionalidad requerida actualmente para el sistema.

El diseño en XP no es una actividad que se realice una única vez al comienzo del desarrollo. Hay pasos de diseño en cada Planificación de Entregas y Planificación de cada Iteración. El diseño es un proceso incremental e iterativo y se lleva a cabo por revisiones periódicas y refactorización.

Programación de a Pares

Todo el software de producción en XP es realizado por una pareja de programadores, sentados uno al lado del otro frente a la misma estación. Esta práctica asegura que todo el código de producción es revisado por al menos otro programador y esto resulta en un mejor diseño, mejor prueba, y mejor código.

Puede parecer ineficiente tener dos programadores haciendo el trabajo de un solo programador, sin embargo está demostrado lo contrario.

Test-First Development

XP es obsesivo con la retroalimentación, y en desarrollo de software, buena retroalimentación requiere buenas pruebas.

Los equipos XP tops practican "Test-First Development", trabajando en ciclos muy cortos de agregar una prueba y luego realizar el trabajo. Los equipos producen código con casi el 100 % de las pruebas cubiertas.

No es suficiente con escribir las pruebas, hay que ejecutarlas. Aquí también XP es "extrema". Estas "pruebas del programador" o "pruebas de unidad" son recopiladas todas, y cada vez que el programador libera código al repositorio, todas las pruebas deben ser ejecutadas correctamente.

Mejora del Diseño

XP se enfoca en entregar en cada iteración algo de valor del negocio. Para realizar esto en el curso de un proyecto completo, el software debe estar bien diseñado. XP utiliza un proceso de diseño continuo denominado "refactorización". Dicho término proviene del título del libro de Martin Fowler "Refactoring: Improving the Design of Existing Code".

El proceso de refactorización se enfoca en la eliminación de código duplicado incrementando la cohesión y minimizando el acoplamiento del código.

El resultado es que los equipos XP comienzan con un diseño simple y bueno, y lo mantienen a lo largo de todo el diseño.

La refactorización es soportada por supuesto por pruebas intensivas de modo de asegurar que a medida que el diseño evoluciona, no se "rompe" nada de lo hecho.

Integración Continua

Los equipos XP mantienen el sistema integrado todo el tiempo.

La integración poco frecuente conlleva serios problemas en los proyectos de software.

Primero, aunque la integración es crítica para lograr código que trabaje correctamente,

Propiedad Colectiva del Código

En un proyecto XP, cualquier par de programadores puede mejorar cualquier código en cualquier momento. Esto significa que todo el código tiene el beneficio de la atención de mucha gente, lo cual incrementa su calidad y reduce sus los defectos.

La propiedad colectiva del código puede ser un problema si la gente trabaja a ciegas en código que no comprende. XP evita este problema a través de dos técnicas: las *pruebas del programador* para descubrir errores, y la *programación de a pares*, en este caso la mejor manera de trabajar con código no familiar es trabajar de a par con el experto.

Codificación Estándar

Los equipos XP siguen una codificación estándar común, de tal modo que todo el código en el sistema parece ser escrito por la misma persona. Además la codificación estándar favorece la propiedad colectiva del código.

Metáforas

Los equipos XP desarrollan una visión común de cómo trabaja el sistema, que se denomina *metáfora*. En el mejor de los casos, la metáfora es una descripción simple de cómo el programa trabaja el programa, por ejemplo "este programa trabaja como una colmena de abejas, que salen afuera por polen y lo traen de regreso al panal" como una descripción para un sistema de recolección de información basada en agentes.

A veces no aparece una metáfora tan imaginativa. En cualquier caso los equipos XP utilizan sistemas de nombres comunes para asegurarse de que todos comprenden como funciona el sistema, y donde encontrar la funcionalidad que uno busca, o donde ubicar el lugar apropiado para agregar nueva funcionalidad.

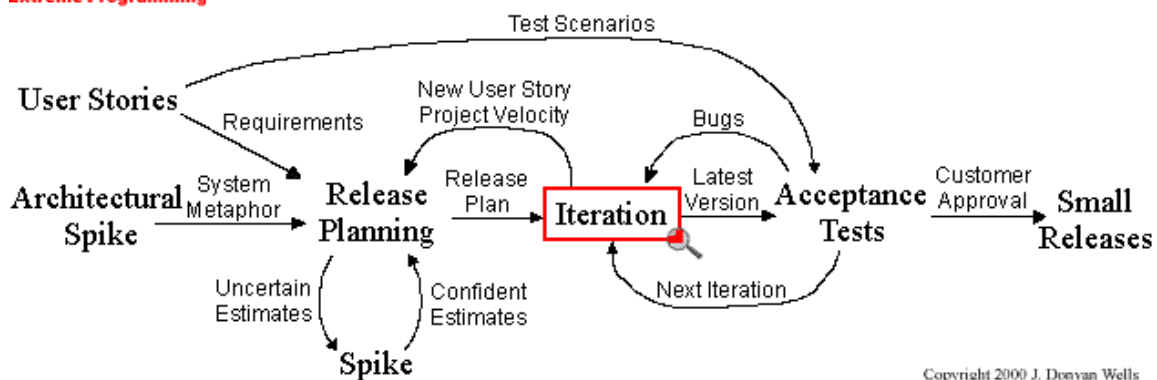
Ritmo Sostenido

Los equipos XP tratan de trabajar a un ritmo continuo evitando los "tiempos muertos".

El Proyecto XP



Extreme Programming Project



Historias de Usuario

Las historias de usuario sirven al mismo propósito que los casos de uso, pero no son la misma cosa. Son utilizadas para realizar estimaciones de tiempo para la *planificación de entregas*.

Las historias de usuario son escritas por los clientes como cosas que el sistema debe hacer para ellos.

Son escritas en formato de un texto en el lenguaje del usuario sin terminología técnica.

Las historias de usuario también son utilizadas para generar las pruebas de aceptación. Una o más pruebas automáticas deben crearse para verificar que una historia de usuario ha sido implementada correctamente.

Las historias de usuario deben tener un nivel de detalle apenas suficiente para realizar una estimación de cuanto tiempo durará su implementación. Cuando llega el momento de implementar la historia de usuario, los desarrolladores se juntan con el usuario y reciben información más detallada en una conversación cara-a-cara.

Típicamente el tiempo necesario para la implementación de una historia de usuario varía de 1 a 3 semanas. Duraciones mayores a 3 semanas implican que debe particionarse la historia de usuario.

Alrededor de 80 + o – 20 historias de usuario son suficientes para crear un plan de entregas razonable.

En una historia de usuario deben evitarse detalles específicos acerca de tecnología, diseño de base de datos, algoritmos, etc.

Las historias de usuario son escritas en tarjetas impresas o manualmente.

Planificación de Entregas (release planning)

Se crea un plan de entregas que es un plan general de todo el proyecto. Luego el plan de entregas es utilizado para desarrollar un plan de iteraciones para cada iteración particular.

La planificación de entregas tiene un conjunto de reglas que permite a todos los involucrados tomar sus propias decisiones.

La esencia de la planificación de entregas radica en la estimación del tiempo necesario para la implementación de cada historia de usuario.

Luego el cliente define cuales son las historias con mayor prioridad a implementar primero.

Las iteraciones individuales son planificadas en detalle inmediatamente antes de su realización, no por adelantado.

Las reuniones de planificación de entregas son llamadas el "juego de planificación".

La base filosófica de la planificación de entregas es que un proyecto puede ser cuantificado por cuatro variables: alcance, recursos, tiempo, y calidad. El alcance se refiere a cuanto trabajo deberá realizarse. Los recursos se refieren a cuantas personas están disponibles. Tiempo, es cuando el proyecto o la entrega será realizada. Calidad es cuán bueno será el sistema y cuan bien probado será.

Planificación de Iteración

Una reunión de planificación de iteración es realizada al comienzo de cada iteración para producir un plan de tareas de programación a realizar durante la iteración.

Cada iteración tiene una duración de entre 1 a 3 semanas. Las historias de usuario son seleccionadas para la iteración por el cliente del plan de entregas. También se seleccionan las pruebas de aceptación fallidas a corregir.

Las historias de usuario y pruebas de aceptación fallidas son trasladadas en tareas de programación, las cuales son escritas en tarjetas. Estas tarjetas constituyen el plan detallado para la iteración.

Cada tarea tiene una duración típica de 1 a 3 días. Las tareas que son más breves que un día pueden agruparse en el mismo día. Tareas de mayor duración a 3 días deben descomponerse en otras menores.

Crear una solución "spike"

Crear soluciones "spike" para representar cuestiones técnicas o de diseño. Una solución "spike" es una solución muy sencilla para experimentar soluciones potenciales. Su objetivo es reducir el riesgo de problemas técnicos o incrementar la confiabilidad de una estimación sobre una historia de usuario.

Pruebas de aceptación

Las pruebas de aceptación son creadas a partir de las historias de usuario. Durante una iteración, las historias de usuario seleccionadas son trasladadas en pruebas de aceptación. Una historia puede tener una o varias pruebas de aceptación.

Las pruebas de aceptación son pruebas de caja negra. Los clientes son responsables de verificar que las pruebas se ejecutan correctamente.

Una historia de usuario no se considera completa hasta que no ha superado satisfactoriamente las pruebas de aceptación.

El aseguramiento de la calidad es una parte esencial de XP. En otros enfoques el aseguramiento de la calidad puede ser realizado por un equipo independiente. En XP se requiere una relación más estrecha.

Pequeñas entregas

El equipo de desarrollo necesita liberar versiones del sistema iterativamente a menudo.

Las reuniones de planificación de entregas son utilizadas para descubrir pequeñas unidades de funcionalidad que tienen importancia para el negocio. Esta práctica es fundamental para obtener una rápida retroalimentación.

Límites de XP

Diversos autores encuentran problemas y limitaciones en la utilización de XP. La mayoría de las observaciones se enfocan en las dificultades de utilización de XP para proyectos de gran envergadura, y su limitación a contextos restringidos. Entre las principales observaciones podemos citar:

- XP propone que la refactorización hace innecesario el análisis de requerimientos tradicional. La flexibilidad es restablecida analizando el código y rediseñándolo para que permanezca lo más simple posible. Este enfoque, aunque interesante, puede no ser aplicable en grandes sistemas. Para sistemas complejos y delicados, los desarrolladores continuarán necesitando especificaciones de requerimientos detalladas, documentos del diseño, y demás, y las autoridades reguladoras, continuarán demandando evidencia de enfoques de desarrollo rigurosos.
- No queda claro como utilizar XP con grandes cantidades de usuarios (cientos o miles). XP parece estar dirigido al desarrollo de software a medida para empresas donde los desarrolladores y usuarios pueden sentarse a conversar alrededor de una mesa. Cuando los requerimientos del usuario deben transmitirse a través de una mesa de ayudas, un departamento de marketing, o un equipo de gestión del producto, XP inmediatamente parece ser muy difícil de mantener.
- XP está dirigido explícitamente al desarrollo de software únicamente. Sistemas que requieran hardware específico, inevitablemente demandan especialistas (ingenieros, ergonomistas, etc.).
- XP tiene un enfoque en la habilidad del equipo de trabajo, donde cada miembro del equipo debe ser un generalista capaz de trabajar en cualquiera de todo el rango de actividades del desarrollo de software. En problemas pequeños que puedan ser manejados por una sola persona o un equipo pequeño, este enfoque puede ser altamente eficiente. Sin embargo la pericia en el desarrollo de software en pequeños equipos, no implica la misma en la construcción de grandes sistemas.

- Por otra parte, este enfoque demanda equipos de personas talentosas, las cuales no son muy abundantes en las organizaciones, en tanto que en ellas se encuentran personas de muy variados niveles. Peor aún, es una característica de las personas talentosas, no querer ocuparse del mantenimiento de sistemas heredados. Las organizaciones enfrentan este problema creando procesos a nivel de empresa que permita a personas menos habilidosas realizar las tareas necesarias colectivamente. Estos procesos son menos eficientes que aquellos basados en personas altamente habilidosas, pero pueden aplicarse cuando no se dispone de número de personas necesarias o los costos no lo permiten.
- Desarrollo centrado en código más que centrado en el diseño: aunque la pérdida de prácticas de diseño en pequeños sistemas pueda no ser problema significativo, en grandes sistemas puede ser un desastre.
- Pérdida de documentación del diseño: limita XP a pequeños programas y dificulta las oportunidades de reutilización.
- La producción de código fácilmente legible (forma de documentar de XP), ha sido un objetivo largamente anhelado y no alcanzado en los pasados cuarenta años. Más aún, utilizar código fuente para documentar grandes sistemas es impráctico ya que los listados pueden contener cientos de páginas.
- Pérdida del proceso de revisión estructurada: cuando los desarrolladores revisan sus programas en pantalla, encuentran un 10-25% de errores. Aún con programación de pares, no se supera el 40%.
- Calidad a través de la prueba: un proceso de desarrollo que depende fuertemente de la prueba es improbable que produzca productos de calidad. La pérdida de un proceso ordenado de diseño y el uso de revisiones no estructuradas, significa la necesidad de intensivos procesos de prueba consumidores de tiempo.
- Pérdida de un plan de calidad: XP no planea, mide, o gestiona explícitamente un plan de calidad.
- XP no provee guías explícitas en la captura de datos.
- Los métodos son descriptos brevemente: cuando los métodos de ingeniería son descriptos muy brevemente, los practicantes adoptan las partes que les gusta y descartan las que no.
- Falta de soporte para la transición: la transición a un nuevo proceso o nueva tecnología en general es una tarea desafiante. Una transición exitosa requiere considerables recursos, un programa de soporte a largo plazo, y medición y análisis del esfuerzo. XP no provee soporte para esto.

Conclusiones

Los métodos ágiles constituyen un enfoque controvertido y esperanzador a su vez, para la solución de uno de los mayores problemas que enfrenta el desarrollo de software: el cambio continuo.

En la experiencia profesional observamos dos situaciones clásicas, organizaciones con muy poco o ninguna utilización de metodologías, donde el desarrollo de software se realiza de forma artesanal y las prácticas obedecen a capacidades y formaciones particulares de los individuos y la tradición oral de la organización. En el otro extremo se presenta el caso de organizaciones que han adoptado alguna metodología de las tradicionales, ya sean académicas o productos comerciales. Sin embargo en estos casos es habitual ver que se invierte muchos recursos en la creación de documentos que finalmente en su gran mayoría quedan desactualizados y por tanto sin valor. Más aún cuando sobrevienen los cambios y la urgencia por dar soluciones, lleva a que no se continúe actualizando los modelos y documentación del sistema original. Otro inconveniente observado es la burocracia de estas metodologías que puede llevar a dos problemas: demasiado tiempo hasta que el usuario tenga algún software funcionando, y por otra parte lentitud para dar respuesta a los cambios dinámicos de los requerimientos.

Los métodos ágiles con su enfoque iterativo e incremental, con pequeñas entregas reales a corto plazo, y con el desarrollo y mantenimiento de modelos mínimos y sencillos, tiende a solucionar los problemas planteados.

Sin embargo creemos que existen cuestiones que quedan por considerar:

- ▼ El enfoque altamente iterativo e incremental de los métodos ágiles los hace sumamente aptos para entornos de cambio de requerimiento muy dinámicos, tal como el software propietario para gestión de negocios. Sin embargo, consideramos una diversidad de casos donde este enfoque puede no ser el más apto, donde se pueden tener requerimientos claramente definidos desde un comienzo, y donde un diseño detallado y minucioso puede/debe aplicarse convenientemente. Tal lo vemos en casos como:
 - Sistemas técnicos
 - Sistemas de control en tiempo real (sistemas de salud, controles industriales, controles de aeronaves, controladores de armamentos, etc.)
 - Paquetes de software específicos (procesadores de palabra, planillas, graficadores, etc.)
 - Sistemas ERP altamente configurables (Ej. SAP)
 - Frameworks de diferente naturaleza
- ▼ Algunas prácticas, como la programación de a pares, puede no ser adoptada por los gerentes por considerarse un mal uso de recursos. Particularmente en muchas empresas del medio, difícilmente paguen dos salarios para realizar el trabajo de uno.
- ▼ Tal como se explicó en el desarrollo del trabajo, los métodos ágiles por su desarrollo incremental e iterativo, no pueden sostener una planificación detallada desde un comienzo, lo cual hace que no sean aptos para el establecimiento de Presupuestos Fijos antes del inicio del proyecto. Este si bien es un planteo realista en muchos casos, difícilmente sea aceptado por muchos gerentes y organizaciones que demandan presupuestos claros y fijos antes de embarcarse en un proyecto.
- ▼ Sistemas comerciales ERP. Cada día se ve con mayor frecuencia que las empresas adoptan paquetes de software ya elaborados para la gestión de la empresa (Ej. SAP, JDE, PeopleSoft, etc.), en vez de desarrollar sus propias soluciones a medida. Vemos de difícil implementación los métodos ágiles en estos casos. Particularmente no sería sencillo aplicar prácticas de simplicidad y refactorización, en productos cerrados que demandan el cumplimiento de una gran cantidad de reglas para su adaptación a un entorno específico.

- ▼ Muchas empresas de software grandes, han distribuido geográficamente sus entornos de desarrollo. Muchas de ellas llevan los entornos de programación a países donde les resulta más económico (tal el caso de India o Latinoamérica). En estos casos los programadores trabajan bajo pedidos concretos, a gran distancia de los usuarios o de otros miembros de equipo. Vemos de difícil instrumentación en estos casos los métodos ágiles, que demandan una comunicación muy personalizada y uso de pocos documentos para transmitir los requerimientos.
- ▼ Refactorización. Esta práctica central y fundamental para el diseño evolutivo, solo puede ser aplicada eficientemente con la tecnología apropiada. Particularmente XP como los otros métodos están fuertemente orientados a la tecnología de objetos, en el caso de XP tiene fuertes vínculos con Smalltalk. Sin embargo vemos de difícil a imposible su aplicación en una gran cantidad de entornos actualmente en vigencia, como por ejemplo los basados en mainframe con lenguajes y sistemas de datos tradicionales donde aplicar modificaciones no es tan sencillo. Este problema se agrava más aún en sistemas grandes con grandes volúmenes de datos. No olvidemos que precisamente uno de los objetivos de las bases de datos relacionales es facilitar la modificación de los esquemas de datos causando el mínimo impacto en los programas.

Finalmente, sí vemos conveniente el enfoque de los métodos ágiles para el caso de sistemas de gestión de negocios propietarios, con un alto compromiso y participación por parte del usuario, con equipos de trabajo pequeños (no más de 10 personas), y con la tecnología apropiada.